Practical Assessment-3:-Deployment Plan

Three-Tier Application Deployment Plan

Company: Modern India Pvt Limited
Project: Full Stack Web Application Deployment
Role: DevOps Engineer (Intern)
Technology Stack: React, Node.js, MySQL
Cloud Platform: AWS

---

1. Introduction

1.1 Purpose

The purpose of this deployment plan is to describe the design, architecture, and implementation strategy for deploying a **three-tier web application** on AWS infrastructure.

The deployment follows **industry best practices for security, monitoring, scalability, and reliability**.

The plan ensures that the application:

- Runs in a **secure AWS environment**
- Supports **end-to-end communication between frontend, backend, and database**
- Implements **monitoring and alerting**
- Uses **HTTPS for secure access**
- Provides **observability through dashboards and logs**

1.2 Scope

This deployment includes:

- Infrastructure setup on AWS
- Deployment of the three-tier application
- Monitoring and observability implementation
- Security configuration
- Alerting and notification setup
- Documentation and validation

This deployment will include:

- Frontend deployment
- Backend API deployment
- MySQL database setup
- Monitoring stack deployment
- HTTPS configuration
- Alert notifications

The deployment will **not include application development**, as the source code is already provided in the GitHub repository.

GitHub Repository:

```
https://github.com/pushkar-iamops/3tier-monitoring
```

---

2. Application Architecture Overview

The system follows a **three-tier architecture** which separates application functionality into independent layers.

This architecture improves:

- Security
- Maintainability
- Scalability
- Fault isolation

Three Layers

Layer	Technology	Purpose
Frontend Layer	React	User interface
Application Layer	Node.js API	Business logic
Data Layer	MySQL	Data storage

Application Flow

```
User
 ↓
React Frontend
```

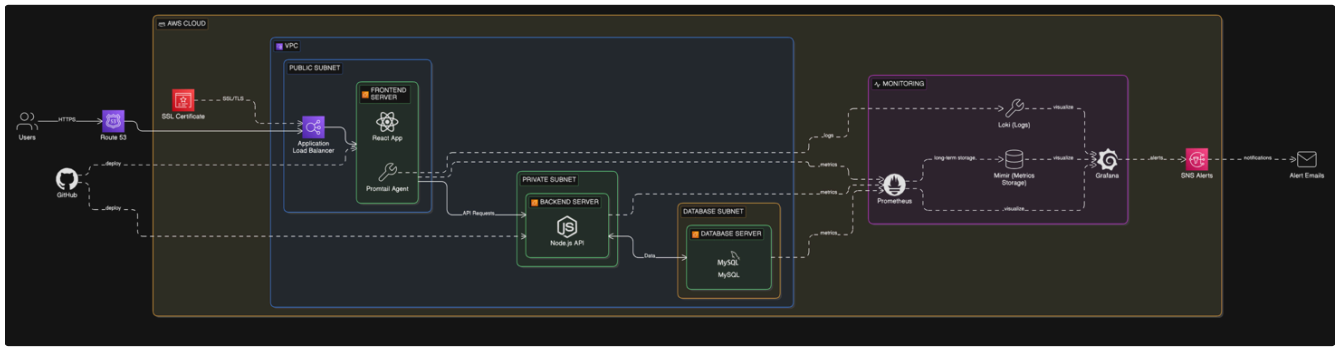
```
 ↓
Node.js Backend API
 ↓
MySQL Database
```

Users interact with the **React application**, which communicates with the **Node.js backend API**, and the backend retrieves or stores data in the **MySQL database**.

---

3. Infrastructure Design

The infrastructure will be deployed on **Amazon Web Services (AWS)** using **EC2 instances and a custom VPC architecture**.



Core Infrastructure Components

Component	Purpose
VPC	Isolated network environment
EC2 Instances	Host application services
Security Groups	Network access control
Internet Gateway	Internet access
Monitoring Stack	Observability and metrics

EC2 Instances

Three EC2 instances will be created.

Instance	Purpose
Frontend Server	Hosts React application
Backend Server	Runs Node.js API

Database Server	Runs MySQL database

Each instance will run **Ubuntu Linux**.

---

4. Networking Architecture

A **custom VPC** will be created to isolate the infrastructure.

Network Components

- VPC
- Public subnet
- Private subnet
- Internet gateway
- Route tables

Network Layout

```
VPC
 ├── Public Subnet
 │       └── Frontend EC2
 │
 └── Private Subnet
         ├── Backend EC2
         └── Database EC2
```

Reasoning

- Frontend server must be **accessible to users via internet**
- Backend server should **not be directly exposed**
- Database must be **private and secure**

---

5. Security Design

Security will be implemented using **AWS Security Groups** and restricted network access.

Security Groups

Frontend Security Group

Port	Access
22	Admin IP
80	Public

443	Public

Purpose:
Allows users to access the frontend application securely.

Backend Security Group

Port	Access
22	Admin IP
3000	Frontend EC2

Purpose:
Only frontend server can communicate with backend API.

Database Security Group

Port	Access
3306	Backend EC2

Purpose:
Database can only be accessed by backend server.

---

6. Application Deployment Strategy

The application will be deployed from the GitHub repository.

Repository:

```
pushkar-iamops/3tier-monitoring
```

6.1 Frontend Deployment (React)

The React application will be deployed on the frontend EC2 instance.

Deployment Steps

1. Install Node.js and npm

2. Clone repository
3. Build React application
4. Serve using Nginx

Command flow:

```
git clone repository
cd frontend
npm install
npm run build
```

The built application will be served using **Nginx** for improved performance.

Why Nginx?

- Fast static file serving
- Production ready
- Handles high traffic efficiently

6.2 Backend Deployment (Node.js API)

The backend API will be deployed on the backend EC2 instance.

Deployment Steps

1. Install Node.js
2. Clone repository
3. Install dependencies
4. Run backend application

Command flow:

```
git clone repository
cd backend
npm install
node app.js
```

Process Management

The application will be managed using **PM2**.

Benefits of PM2:

- Automatic restart on failure
- Background execution
- Process monitoring

6.3 Database Deployment (MySQL)

MySQL will be installed on the database EC2 instance.

Deployment Steps

1. Install MySQL server
2. Create database
3. Configure user access
4. Allow backend server connection

Example configuration:

```
CREATE DATABASE appdb;
CREATE USER 'appuser';
GRANT ALL PRIVILEGES ON appdb TO 'appuser';
```

The database will only accept connections from the **backend EC2 instance**.

---

7. Monitoring and Observability

Monitoring ensures system reliability and performance.

The monitoring stack will include:

Tool	Purpose
Prometheus	Metrics collection
Mimir	Long-term metrics storage
Loki	Log aggregation
Grafana	Dashboard visualization

7.1 Metrics Collection

Metrics will be collected from:

- Frontend EC2
- Backend EC2
- Database EC2

Using **Node Exporter**.

Metrics include:

- CPU usage
- Memory usage
- Disk usage
- Network usage

7.2 Grafana Dashboards

Grafana will visualize the collected metrics through dashboards.

Dashboards will display:

- CPU utilization
- Memory consumption
- Disk usage
- System health

These dashboards will help administrators monitor system performance.

7.3 Log Aggregation

Logs from the **frontend EC2 instance** will be collected using:

- Promtail
- Loki

Logs will be visualized inside **Grafana**.

This enables centralized log monitoring.

---

8. Alerting and Notifications

Alerting will be implemented to notify administrators when system issues occur.

Alert Conditions

Alerts will trigger when:

- CPU usage exceeds threshold
- Memory usage is high
- Disk usage is high

- Application becomes unavailable

Alert Flow

```
Prometheus
 ↓
Alertmanager
 ↓
AWS SNS
 ↓
Email Notification
```

SNS will send **email alerts to administrators**.

---

9. HTTPS Security Implementation

The application will be secured using **HTTPS**.

SSL certificates will be generated using **Let's Encrypt**.

Tool used:

```
Certbot
```

HTTPS ensures:

- Encrypted communication
- Secure data transfer
- Production-grade security

Users will access the application via:

```
https://application-domain
```

---

10. Continuous Deployment

Continuous deployment will be implemented using **automation scripts**.

Deployment script will automate:

- Pull latest code from GitHub
- Build application
- Restart services

This improves deployment efficiency and reduces manual errors.

---

11. Validation and Testing

After deployment, the system will be validated.

Validation includes:

- Application accessibility
- Frontend to backend communication
- Backend to database connectivity
- Monitoring functionality
- Alert notifications
- HTTPS security

---

12. Expected Deliverables

The final submission will include:

Deployment Plan

Detailed description of architecture and deployment strategy.

Architecture Diagram

Diagram showing AWS infrastructure components and communication flow.

Deployment Documentation

Screenshots of:

- Running application
- Database validation
- Prometheus targets
- Grafana dashboards
- Loki logs
- SNS alert email
- HTTPS enabled application

---

13. Conclusion

This deployment plan provides a structured approach to deploying a **three-tier application on AWS** using DevOps best practices.

The architecture ensures:

- Secure communication
- Scalable infrastructure
- Comprehensive monitoring
- Reliable alerting system

By implementing this design, the application will operate in a **production-ready cloud environment** with proper observability, security, and reliability.